Getting Started

Getting Started · docs/latest · generated 2026-08-02

This guide covers the three ways to work with StarUI: consuming the packages in **your own application**, running the **demo apps**, and developing the **platform itself**.

---

1. Consume the packages in your app

The packages install as ordinary npm packages under their real names. From a registry (Artifactory) that hosts them:

```
npm install @wellsfargo-starui/grid @wellsfargo-starui/react \
            @wellsfargo-starui/design-system
```

Without a registry, install the packed tarballs directly — `npm run pack:npm` in the platform repo emits one tarball per package under `dist-npm/`:

```
npm install /path/to/dist-npm/wellsfargo-starui-grid-0.1.0.tgz
```

> The packed tarballs depend on each other by version. Installing from files with no registry needs an `overrides` block naming the whole set — see the generated `apps/tarball/*/package.json` for the working shape.

Your app owns the framework peers:

```
npm install react react-dom ag-grid-community ag-grid-enterprise \
            ag-grid-react @tanstack/react-query tailwindcss
```

A minimal grid host

The `apps/source/basic` app is the canonical tutorial — a complete bond blotter in a few files. The essential wiring:

```
import {
  MarketsGrid,
  createMarketsGridLocalStorageStorage,
} from '@wellsfargo-starui/grid';
import { applyTheme, getTheme } from '@wellsfargo-starui/design-system';
import '@wellsfargo-starui/design-system/styles.css';
import '@wellsfargo-starui/grid/styles.css';

const storage = createMarketsGridLocalStorageStorage();

export function App() {
  return (
    <MarketsGrid
      gridId="bond-blotter-v1"      // stable id — keys profile persistence
      rowData={rows}
      columnDefs={columnDefs}
      defaultColDef={defaultColDef}
      rowIdField="id"
      storage={storage}             // where profiles + grid state persist
      showFiltersToolbar
      showFormattingToolbar
      showEditingToolbar
    />
  );
}
```

What you get out of the box: profile persistence (columns, filters, formats survive reload), the filters / formatting / editing toolbars, the column customizer, and full dark/light theming.

Theming

Themes are design-system tokens switched by one attribute:

```
import { applyTheme, getTheme } from '@wellsfargo-starui/design-system';

applyTheme({ theme: 'dark' });          // flips data-theme on <html>
const { theme } = getTheme();           // 'dark' | 'light'
```

Never hardcode colors — consume `--bn-*` / `--fi-*` CSS variables or the semantic exports from `@wellsfargo-starui/design-system/tokens/semantic`. Every surface must render correctly under both `data-theme="dark"` and `data-theme="light"`.

UI primitives

Build app chrome from `@wellsfargo-starui/react` (shadcn/Radix primitives pre-wired to the tokens) — not native `<input>` / `<select>` / `<textarea>`:

```
import { Button, Tooltip, TooltipTrigger, TooltipContent } from '@wellsfargo-starui/react';
```

Live data (optional)

To feed grids from a shared upstream connection, add `@wellsfargo-starui/data`: the SharedWorker owns one STOMP session per browser profile and fans snapshots + thin deltas out to every window. See architecture.md § data services →.

---

2. Run the demo apps

The demos live in this repo under `apps/` — **their own npm install root**, separate from the package workspaces:

```
npm install && npm run build

cd apps
npm install
npm run typecheck && npm run build     # both consumption tracks

cd source/basic && npm run dev         # any single app
```

APP	PORT	PURPOSE
`star-demo`	5175	OpenFin workspace demo; primary e2e target
`markets-grid-lab`	5300	MarketsGrid editing / profiles lab
`design-system`	5310	design-system showcase
`stomp-marketsgrid-minimal`	5213	smallest STOMP → grid path
`basic`	5194	tutorial — minimal grid host
`dataprovider-editor`	5193	tutorial — data-provider editor
`stomp-view-server`	8081	STOMP fixture server (not a UI)

Each UI app also has a generated **tarball twin** (same code, consuming vendored tarballs, port +1000). Regenerate the twins after package changes:

```
cd apps
npm run setup:tarball        # vendor pack:npm output first …
npm run make:tarball-apps    # … regenerate the twins …
npm run setup:tarball        # … and install them
```

---

3. Develop the platform

```
npm install            # workspaces: packages/* only
npm run build          # turbo build + consumer-tsconfig regeneration
npm run typecheck      # build, then turbo typecheck
npm test               # Vitest across packages/
npm run lint:all       # eslint + cycles + token rules + RTL check
```

Before shipping a change:

```
npx turbo typecheck build test          # the green bar
npm run test:coverage                   # per-file coverage run
npm run check:coverage                  # 70%-per-file gate
```

Conventions that will save you a review round-trip:

- **Coverage is per file** — 70% on lines, statements, functions and branches, with `all: true`, so a new untested file fails the gate at 0%.
- React components are tested with **React Testing Library** (`npm run check:rtl` enforces it).
- Filenames match the case of the primary export; folders are kebab-case.
- Update `docs/current-features.md` in the same change as any feature add/update/remove.
- Never `npm ci`, never `pnpm` / `yarn`, no `--legacy-peer-deps` — plain `npm install` must always resolve cleanly.

The full agent/contributor rulebook is CLAUDE.md; the architecture and layer rules are in architecture.md →.